GOULOT · KIT DE CONSTRUCTION

Goulot — instructions pour Claude Code

Version du 10 octobre 2026 · texte de référence pour Claude Code : `CLAUDE.md`

Goulot est une app personnelle de veille des goulots d'approvisionnement, installée sur le Mac de son utilisateur. À partir d'un nom de matière ou d'un numéro CAS, elle reconstruit la chaîne d'approvisionnement théorique, surveille chaque maillon avec des sources publiques gratuites, et donne un verdict par matière : « Rien à signaler », « Problème potentiel à X semaines » ou « Anticiper stock ». L'usage est l'achat : sécuriser du stock avant une hausse ou une pénurie.

L'utilisateur peut aussi déclarer sa chaîne réelle, « Ma chaîne », qui affine les verdicts : elle reste chiffrée sur le Mac, derrière un coffre ouvert par Touch ID, et n'est jamais envoyée à l'IA. Principe : l'IA juge le marché, le Mac juge l'exposition de l'utilisateur.

L'utilisateur n'est pas développeur. Quand tu as besoin de lui, explique en une phrase simple ce que tu fais et ce que tu attends.

À lire avant de coder

- `docs/SPEC.md` : fonctionnement détaillé (données, pipeline, verdict, écrans, sources).
- `docs/SECURITY.md` : modèle de menaces et contrôles obligatoires. Non négociable.
- `docs/UPDATES.md` : mise à jour automatique signée, avec retour arrière.
- `docs/DECISIONS.md` : décisions prises. Ajoute-y chaque nouvelle décision, datée et justifiée.
- `prompts/` et `schemas/` : prompts et schémas JSON des appels IA. Versionnés : toute modification crée un nouveau fichier `_v2`, jamais une réécriture silencieuse. `schemas/examples/` contient un exemple fictif par schéma ; `schemas/check_schemas.py` vérifie les contraintes de l'API, les exemples et les contrôles métier à appliquer à chaque sortie réelle.
- `config/` : sources de données et tarifs de l'API, à vérifier en phase 1 ; fiabilité des sources ; modèles d'export de conformité (`config/compliance/`).
- `design/` : maquette de référence, faite dans un outil de design. Sers-t'en pour la mise en page, les couleurs et les textes, pas comme code. Ses données sont fictives.
- `LISEZMOI.pdf`, `PREMIER_MESSAGE.txt` et `A_LIRE/` : copies de lecture pour l'utilisateur, qui ne lit pas le Markdown. Les fichiers `.md` font foi ; ne modifie pas les PDF. Quand tu changes un document qu'il doit lire, dis-le-lui en une phrase dans ton résumé de phase.

Règles non négociables

1. **Réseau.** Le serveur n'écoute que l'interface de bouclage (`127.0.0.1`, et `::1` seulement si un navigateur l'exige). Il refuse de démarrer si une autre adresse est configurée. L'interface est servie sur `http://localhost:8742`, car WebAuthn refuse les adresses IP. Pas de CORS.
2. **Secrets.** Uniquement dans le trousseau macOS, via `keyring` (service `goulot`). Jamais dans le code, un fichier `.env`, la base, les journaux, le frontend, git ou la conversation. Ne demande jamais à l'utilisateur de coller une clé dans le chat : il passe par `goulot setup`, en saisie masquée.
3. **Contenu externe = donnée non fiable.** Articles, flux, pages web, résultats de recherche et réponses des modèles ne sont jamais exécutés ni traités comme des instructions, et sont toujours affichés comme du texte.

4. **IA sans pouvoir d'action.** Aucun outil à effet de bord : pas de shell, pas d'écriture de fichier, pas d'envoi de message. Les sorties sont du JSON validé par schéma, puis par le code (identifiants existants, bornes, cohérence). Le verdict est calculé par le code, jamais par le modèle.
5. **Données personnelles.** Ma chaîne, signaux fournisseurs, stock, délais, devis, achats, seuils et mesures d'atténuation restent sur le Mac, chiffrés (règle 13), et ne sont jamais envoyés à l'IA : chaque prompt est construit à partir d'une liste blanche de champs. Ils n'apparaissent ni dans une requête vers une source (sauf le nom d'un site que l'utilisateur a autorisé), ni dans une alerte, ni dans un journal. Aucune donnée de l'utilisateur (ses CAS, ses chaînes, ses réglages, ses réponses IA enregistrées) dans le dépôt ni dans les tests : le dépôt peut être public.
6. **Sorties réseau.** Uniquement via le client HTTP maison (`goulot/net/client.py`) : liste blanche de domaines, HTTPS obligatoire, blocage des adresses privées après résolution DNS, tailles et délais limités. Le SDK Anthropic reçoit un client construit sur le même transport (`http_client=`). Aucune bibliothèque qui fait ses propres appels réseau. Deux exceptions : les e-mails d'alerte (SMTP en TLS, mêmes contrôles DNS et d'adresses privées) et `uv sync --locked --no-build` de l'updater, détaillé dans `docs/UPDATES.md`.
7. **Dépendances.** Le moins possible, versions figées avec empreintes (`uv.lock`, `package-lock.json`). Toute dépendance hors de la liste « Stack » ci-dessous est justifiée dans `docs/DECISIONS.md`, après vérification de sa maintenance et de sa réputation, et validée par l'utilisateur. `test_dependencies_allowlist` compare les dépendances directes à cette liste.
8. **Coût.** Chaque appel IA passe par `goulot/llm/`, qui compte les tokens, applique le plafond et utilise le traitement différé (Batch) par défaut.
9. **Mises à jour.** Ne jamais casser `goulot doctor` ni la chaîne de signature. Toute version doit pouvoir revenir en arrière.
10. **Langue.** Code, identifiants, commits et commentaires en anglais ; interface, alertes, messages d'erreur et docs utilisateur en français.
11. **Tests.** Aucune tâche n'est terminée si `tests/security` ou la CI échouent.
12. **Registre.** Le registre des conseils (`goulot/ledger/`) est en ajout seul, chaîné par empreintes : aucune fonction, aucune migration ne modifie ou ne supprime une entrée. Les statistiques de fiabilité se calculent uniquement depuis le registre (`docs/SPEC.md`, section 7). Le silence ne compte jamais comme une réussite : sans rien d'observable, un résultat est « non résolu ».
13. **Coffre.** Aucune donnée privée n'est écrite en clair, où que ce soit : base, journal WAL, sauvegarde, journal, fichier temporaire, URL. Toute route de `/api/private/` exige un jeton de coffre obtenu par clé d'accès (Touch ID). La CLI ne lit jamais les données privées. Chiffrement AES-256-GCM de `cryptography` uniquement, jamais d'algorithme maison. Détails : `docs/SPEC.md`, section 11, et `docs/SECURITY.md`.

Décisions validées (4 et 5 octobre 2026)

Sujet	Décision
Matières de départ	3 numéros CAS, saisis par l'utilisateur dans l'app
Usage	achats
Installation	Mac personnel ; vérifie la version de macOS et le processeur
Modèles	tri : `claude-haiku-4-5-20251001` ; analyse, reconstruction, synthèse : `claude-sonnet-5-5` ; modèle open source local envisagé après le MVP

Sujet	Décision
Alertes	Telegram ou e-mail, au choix dans les réglages
Carte	2D (MapLibre GL JS et tuiles OpenFreeMap) ; pas de globe 3D dans le MVP
Plafond IA	veille : 5 $ par mois, mode économie à 80 %, règles seules à 100 % ; reconstructions : plafond séparé de 10 $ par mois, devis et confirmation avant chaque lancement ; limite dure de 15 $ par mois sur l'espace de travail Anthropic « goulot », qui contient la clé de l'app
Dépôt GitHub	public recommandé (protections gratuites, aucune donnée personnelle dans le code) ; à confirmer avec l'utilisateur à la création ; si privé, signature sur le Mac (`docs/UPDATES.md`)
Données utilisateur	stock, délai fournisseur et Ma chaîne facultatifs
Fiabilité	registre des conseils en ajout seul, chaîné par empreintes et ancré chaque semaine ; écran Fiabilité avec le prix réel, les conseils datés et leurs fourchettes en éventail, pas de courbe de « prix anticipé » ; résultats « non résolus » quand rien n'est observable, colonnes « Marché » et « Ma matière », 90 jours de rodage
Conformité	export de l'évaluation des risques d'approvisionnement sur le modèle de l'article 24 du règlement (UE) 2024/1252, produit localement sans IA (`docs/SPEC.md`, section 10) ; cartographie fondée sur la chaîne type ou sur Ma chaîne
Ma chaîne	chaîne réelle de l'utilisateur en surimpression de la chaîne type ; risque physique sur ses chemins, prix sur le marché ; un site qui pèse pour lui déclenche l'analyse ; signaux fournisseurs (`docs/SPEC.md`, section 11)
Prix	sources officielles et gratuites d'abord (bourses chinoises, EIA, statistiques chinoises, douanes, BLS, Eurostat, Banque mondiale) ; Yahoo et Stooq en dépannage ; tout l'historique disponible ; pression de prix même sans cotation (`docs/SPEC.md`, sections 5 et 6)
Cas comparables	bibliothèque de cas passés préparés par l'IA et vérifiés par l'utilisateur ; réactions des prix calculées par le code, jamais par l'IA ; nombre de cas et fourchette toujours affichés (`docs/SPEC.md`, section 6)
Coffre	données privées chiffrées (AES-256-GCM), clé maîtresse dans le trousseau, clé de secours imprimée, ouverture par clé d'accès (Touch ID) ; mode standard dans le MVP, mode renforcé après

Architecture

```
Navigateur ──(localhost, bouclage uniquement)──▶ FastAPI ──▶ SQLite (WAL ; données privées
chiffrées)
                                           │
APScheduler, toutes les 3 h ──▶ collecteurs ──▶ filtre par règles ──▶ tri IA (Batch)
        ──▶ regroupement et corroboration ──▶ analyse IA (Batch) ──▶ moteur de verdict (code)
        ──▶ registre (ajout seul, empreintes chaînées) ──▶ résultats à l'échéance
        ──▶ tableau de bord, écran Fiabilité, alertes Telegram ou e-mail

Historique des prix + bibliothèque de cas ──▶ réactions calculées par le code ──▶ cas
comparables ──▶ analyse IA, écran
Ma chaîne (chiffrée) ──▶ rapprochement local avec les événements ──▶ exposition ──▶ verdict
personnel
        (aucune donnée privée vers l'IA, les sources, les alertes ou les journaux)
Coffre : clé d'accès (Touch ID) ──▶ jeton de coffre ──▶ routes /api/private/ ; fermeture
automatique

Export de conformité : chaîne validée + événements + registre ──▶ rapport PDF, JSON, CSV (sans
IA)

Reconstruction : identification (PubChem, Wikidata, ECICS) ──▶ recherche web Claude
        ──▶ structuration JSON ──▶ recoupement ──▶ validation par l'utilisateur ──▶ règles de
veille

Updater (LaunchAgent quotidien) ──▶ release GitHub signée ──▶ vérification ──▶ installation
        ──▶ test de santé ──▶ bascule, ou retour arrière automatique
```

Stack

- Python 3.12 ou plus, géré par `uv`, projet non empaqueté (`[tool.uv] package = false`). FastAPI et Uvicorn ; SQLite en mode WAL avec migrations SQL numérotées ; APScheduler ; httpx ; pydantic v2 ; SDK `anthropic` ; `keyring` ; `defusedxml` ; `nh3` ; `networkx` ; `searoute` ; `jsonschema` ; `cryptography` (vérification Ed25519, AES-256-GCM, HKDF) ; `webauthn` (Duo Labs : vérification des clés d'accès du coffre) ; `pyyaml`. Données embarquées : frontières Natural Earth ; liste des villes de plus de 15 000 habitants de GeoNames (licence CC BY 4.0), réduite au nom, aux noms alternatifs, au pays et aux coordonnées.
- Frontend : React, Vite et TypeScript ; TradingView Lightweight Charts (prix) ; MapLibre GL JS avec tuiles OpenFreeMap (carte) ; React Flow (graphe de chaîne) ; polices IBM Plex auto-hébergées, sans appel à Google Fonts.
- Qualité : `ruff`, `mypy` (strict sur `security/`, `net/`, `llm/`, `updater/`), `pytest`, `vitest`, `bandit`, `pip-audit`, `npm audit`, `gitleaks` en pré-commit et en CI.
- Prends la dernière version stable de chaque outil au moment de coder, puis fige-la.
- Les tests n'utilisent jamais le vrai trousseau ni le réseau : `keyring` factice, transport HTTP simulé, réponses IA enregistrées.

Arborescence cible

```
goulot/                      # copie de travail : ~/Projects/goulot
├── CLAUDE.md
├── docs/  prompts/  schemas/  config/  design/
├── pyproject.toml   uv.lock
├── goulot/
│   ├── __main__.py          # CLI : setup, start, open, doctor, update, backup, panic, dev
│   ├── settings.py
│   ├── security/            # session, CSRF, contrôle Host et Origin, en-têtes, secrets,
journal expurgé
│   ├── net/                 # client HTTP et transport à liste blanche
│   ├── db/                  # connexion, migrations/*.sql, sauvegardes
│   ├── chain/               # identification, recherche, structuration, recoupement, délais
│   ├── collect/             # un module par source
│   ├── pipeline/            # filtre, tri, regroupement, corroboration, analyse, verdict,
réconciliation
│   ├── llm/                 # client Anthropic, Batch, budget, construction des prompts,
validation des sorties
│   ├── prices/              # cotations gratuites et indices proxy
│   ├── alerts/              # telegram.py, email.py, neutralisation des liens
│   ├── ledger/              # registre en ajout seul, empreintes, règles de résultat,
indicateurs
│   ├── history/             # historique des prix, liens matière-séries, marge, bibliothèque
de cas, réactions, ressemblance
│   ├── vault/               # clés, chiffrement des champs, clé de secours, clés d'accès
(WebAuthn), jetons de coffre
│   ├── private/             # Ma chaîne, signaux fournisseurs, rapprochement local,
exposition, verdict personnel
│   ├── compliance/          # modèles d'export, calculs (HHI, repère 65 %), rendu, annexes
│   ├── api/                 # routes FastAPI
│   └── updater/             # trust.py, vérification, installation, retour arrière
├── tests/                   # dont tests/security/ et tests/fixtures/
├── frontend/
├── ops/launchd/             # modèles de LaunchAgents
└── .github/                 # workflows/ci.yml, workflows/release.yml, dependabot.yml
```

Appels IA

Vérifie chaque paramètre dans la documentation officielle à jour avant de coder (docs.claude.com ou platform.claude.com) ; note les écarts dans `docs/DECISIONS.md`.

Usage	Modèle	Réglages	Mode
Tri des articles candidats	`claude-haiku-4-5-20251001`	prompt `prompts/triage_v1.md`, `output_config.format` avec `schemas/triage_v1.json`, sans réflexion	Batch
Analyse d'impact	`claude-sonnet-5-5`	prompt `prompts/impact_v1.md`, `thinking: {"type": "adaptive"}`, `output_config.effort: "high"`, `output_config.format` avec `schemas/impact_assessment_v1.json`	Batch ; API standard si la sévérité préliminaire est ≥ 3
Reconstruction, étape 1 : recherche	`claude-sonnet-5-5`	outil `web_search_20260318`, `max_uses: 20`, prompt `prompts/chain_research_v1.md`	standard, en streaming

Usage	Modèle	Réglages	Mode
Reconstruction, étape 2 : structuration	`claude-sonnet-5-5`	prompt `prompts/chain_structure_v1.md`, `thinking: {"type": "adaptive"}`, `output_config.effort: "medium"`, `output_config.format` avec `schemas/supply_chain_v1.json`	standard
Cas comparables, étape 1 : recherche	`claude-sonnet-5-5`	outil `web_search_20260318`, `max_uses: 10`, prompt `prompts/case_research_v1.md`	standard, en streaming
Cas comparables, étape 2 : structuration	`claude-sonnet-5-5`	prompt `prompts/case_structure_v1.md`, `thinking: {"type": "adaptive"}`, `output_config.effort: "medium"`, `output_config.format` avec `schemas/historical_case_v1.json`	standard
Synthèse hebdomadaire	`claude-sonnet-5-5`	prompt `prompts/digest_v1.md`, `thinking: {"type": "between_tools"}` (sans outil, donc sans réflexion), `output_config.effort: "medium"`, `output_config.format` avec `schemas/digest_v1.json`	Batch

Points d'attention connus en octobre 2026 :

- Les sorties structurées sont incompatibles avec les citations, et la recherche web renvoie toujours des citations. D'où la reconstruction en deux appels : recherche libre, puis structuration sans outil. Le code remplace chaque citation par un repère `[srcN]` et garde les URL dans la table `sources` ; l'appel 2 ne voit ni ne produit d'URL.
- Schémas : `additionalProperties: false` partout ; pas de `minimum`, `maximum`, `minLength` ; `minItems` à 0 ou 1 seulement ; au plus 24 champs optionnels et 16 champs à type union par requête. La casse des énumérations n'est pas garantie : compare sans tenir compte de la casse.
- Gère les `stop_reason` `refusal`, `max_tokens` et `pause_turn`. Valide localement chaque sortie avec `jsonschema`. Journalise l'identifiant de requête, jamais le contenu complet.
- Au-delà d'environ 21 000 tokens de sortie, le SDK Python impose le streaming.
- Sur Sonnet 5.5, la réflexion est active par défaut et `thinking: {"type": "disabled"}` renvoie une erreur 400 : pour s'en passer, `between_tools`.
- Cache de prompt d'une heure (`cache_control` avec `ttl: "1h"`) sur les préfixes statiques : minimum 512 tokens sur Sonnet 5.5, 4 096 sur Haiku 4.5.
- Tarifs dans `config/pricing.yaml`, relevés le 3 octobre 2026 : le compteur de coût en dépend, revérifie-les.

Budget (`goulot/llm/budget.py`)

- Coût de chaque appel = tokens réellement facturés × tarif, plus les recherches web. Enregistré dans la table `llm_calls`.
- Veille : plafond mensuel de 5 $, réglable. À 80 % : mode économie (seulement les événements corroborés de sévérité préliminaire ≥ 2, un lot Batch par jour, pas de synthèse). À 90 % : réserve de crise, seuls les événements corroborés de sévérité préliminaire ≥ 3, ou ≥ 2 s'ils touchent un site qui pèse pour l'utilisateur, sont analysés. À 100 % : règles seules et alerte à l'utilisateur ; si un événement de sévérité ≥ 3 est en cours, l'app propose de relever le plafond de veille pour le mois,

dans la limite des 15 $ de l'espace de travail, dépenses de reconstruction du mois déduites. L'utilisateur valide dans l'app ; rien ne se relève seul.

- Reconstruction : devis avant lancement (recherches maximales × 0,01 $ + tokens estimés), confirmation obligatoire, plafond séparé de 10 $ par mois. La préparation des cas comparables suit les mêmes règles, sur le même plafond, un lot par matière.
- Disjoncteur : plus de 20 appels en 10 minutes, ou 3 erreurs d'affilée, déclenchent une pause d'une heure et une alerte.
- Construction : en phase 2, budget de test de 12 $ au plus, validé par l'utilisateur, hors plafonds de l'app ; l'utilisateur porte la limite de l'espace de travail à 25 $ pour ce mois, puis la ramène à 15 $. Les réponses des 3 CAS de test sont enregistrées une fois par `goulot dev record` dans `tests/fixtures/llm/`, puis rejouées ; les CAS de l'utilisateur se reconstruisent dans l'app, sans enregistrement dans le dépôt.

Commandes

Depuis la copie de travail pendant la construction ; ensuite par le lanceur `goulot`. Les commandes qui touchent aux clés ou installent l'app (`goulot setup`, `vault recover`, `vault new-recovery-key`, `dev keygen`, `dev bootstrap`, `dev record`) sont lancées par l'utilisateur dans son Terminal, hors du bac à sable : prépare-les, explique-les en une phrase, attends le résultat.

- `goulot setup` : assistant de configuration (clés dans le trousseau en saisie masquée, création du coffre et affichage unique de la clé de secours, canal d'alerte, test des sources). Lancé par l'utilisateur lui-même dans son Terminal.
- `goulot vault status`, `vault lock` ; `goulot vault recover` (restauration avec la clé de secours) et `vault new-recovery-key`, lancés par l'utilisateur dans son Terminal.
- `goulot start`, `open`, `doctor`, `update --check`, `update --rollback`, `backup`, `panic`.
- `goulot history backfill --cas <CAS>` (historique des séries liées) ; `goulot cases build --cas <CAS>` (préparation des cas comparables, avec devis, aussi depuis l'écran « Bibliothèque de cas ») ;
- `goulot ledger verify` (chaîne d'empreintes du registre) ; `goulot export --template article24 --cas <CAS>…` (rapport de conformité, aussi depuis l'écran Conformité).
- `goulot dev keygen`, `dev release`, `dev bootstrap` : clé de signature, publication, première installation (`docs/UPDATES.md`).
- `goulot dev record --cas <CAS>` : reconstruction réelle d'un des 3 CAS de test, réponses IA enregistrées dans `tests/fixtures/llm/` (jamais pour les CAS de l'utilisateur).
- Développement : `uv run pytest`, `uv run ruff check .`, `uv run mypy goulot`, `uv run python schemas/check_schemas.py`, `npm --prefix frontend run dev`, `build` ou `test`.

Plan de construction

Une phase à la fois. À la fin de chacune : tous les tests passent, tu présentes le résultat en 5 lignes, tu attends le feu vert de l'utilisateur.

Phase 1 — Fondations sécurisées

Squelette du dépôt ; CLI ; serveur sur l'interface de bouclage, interface servie sur `http://localhost:8742` ; accès par lien à usage unique puis cookie de session ; contrôles Host et Origin ; jeton CSRF ; en-têtes de sécurité dont une CSP stricte ; secrets via `keyring` ; coffre complet (clé maîtresse dans le trousseau, clés de données, chiffrement des champs privés, clé HMAC du registre, clé de secours affichée par `goulot setup`, `goulot vault recover` et `new-recovery-key`, clé d'accès WebAuthn, jeton de coffre et fermeture automatique, routes `/api/private/`) : la couverture saisie dès la phase 2 est déjà privée ; client HTTP à liste blanche ; SQLite, migrations et sauvegarde quotidienne ; journaux expurgés ; `goulot doctor` ; tests `tests/security` ; CI GitHub Actions ; `gitleaks` en pré-commit ; `schemas/check_schemas.py` transformé en tests. Vérifie aussi les points d'accès de `config/sources.example.yaml`.

Critères : les tests de phase 1 de `docs/SECURITY.md` passent, sans test vide ni ignoré ; une requête avec `Host: evil.example` est refusée ; aucune route de `/api/` ne répond sans session ; aucune valeur canari n'apparaît en clair dans la base, le WAL ou une sauvegarde ; une restauration avec la clé de secours fonctionne sur une copie de la base ; aucune route de `/api/private/` ne répond sans jeton de coffre ; Touch ID fonctionne dans Safari et dans Chrome sur `http://localhost:8742` ; `goulot doctor` est au vert ; le bac à sable de Claude Code est actif (`/sandbox`).

Phase 2 — Reconstruction de chaîne

Écran « Ajouter une matière » ; identification (PubChem, Wikidata, ECICS) ; recherche puis structuration ; recoupement minimal (USGS, UN Comtrade) ; calcul des délais (searoute et valeurs par défaut) ; écran « Chaîne » avec React Flow et correction des maillons ; devis et confirmation.

Critères : pour 3 CAS de test (107-13-1, 554-13-2, 7440-55-3), puis pour ceux de l'utilisateur, la chaîne contient au moins une voie de production, des pays et des sites sourcés, une confiance par maillon, passe les contrôles de `schemas/check_schemas.py`, et le coût affiché reste sous 3 $ par matière ; tests de phase 2 au vert.

Phase 3 — Veille, verdict et alertes

Collecteurs de `config/sources.yaml` ; filtre par règles ; tri ; regroupement et corroboration ; analyse ; moteur de verdict, délai X et sa fourchette ; fraîcheur de la veille ; registre des conseils dès le premier verdict, règles de résultat (dont « non résolu », « résolu sur indice » et la série privée des devis et achats), rodage et écran « Fiabilité » ; prix gratuits et indice proxy ; écrans « Marchés » et « Fiche matière » avec la carte ; alertes et synthèse hebdomadaire ; budget avec réserve de crise ; alerte de vie externe (facultative). C'est la première version utilisable : l'utilisateur s'en sert sur ses matières pendant la suite de la construction.

Critères : le rejeu d'un jeu figé d'articles (`tests/fixtures/`, réponses IA enregistrées) donne les événements et verdicts attendus ; les articles piégés et les faux sites ne changent pas le verdict ; le plafond déclenche bien le mode économie, la réserve de crise puis les règles seules ; sur `tests/fixtures/ledger/`, les règles de résultat retrouvent les réussites, fausses alertes et hausses manquées attendues, un conseil sans prix ni réponse finit « non résolu », et `goulot ledger verify` détecte l'entrée modifiée et l'entrée supprimée ; « Rien à signaler » devient « Non vérifié » après 12 h sans cycle réussi ; tests de phase 3 au vert.

Phase 3 (suite) — historique et cas comparables. Sources de prix officielles de `config/sources.example.yaml` dans l'ordre de la section 6 de `docs/SPEC.md` ; `instrument_links` proposés par la reconstruction et validés par l'utilisateur ; téléchargement de l'historique ; marge ; pression de prix et ses résultats au registre ; bibliothèque de cas (pistes des bases publiques eMARS, GDACS et ReliefWeb, préparation par l'IA en deux appels, devis, vérification par l'utilisateur) ; réactions calculées ; ressemblance ; `<analogues>` dans le dossier d'analyse ; écrans « Cas comparables » et « Bibliothèque de cas ».

Critères : les jeux de `tests/fixtures/history/` donnent les variations propres, médianes et exclusions attendues ; un brouillon IA n'entre ni dans une médiane ni dans un dossier ; une matière sans série reçoit une pression de prix sans pourcentage ; pour les 3 CAS de test, l'historique de leurs séries est téléchargé et au moins 10 cas sont préparés pour un coût affiché sous 3 $ par matière ; tests au vert.

Phase 4 — Ma chaîne et coffre

Éditeur Ma chaîne sur l'écran « Chaîne » (adoption des maillons de la chaîne type, sites absents, import CSV, localisation hors ligne, niveau de surveillance par site) ; vue coffre fermé sans indice et mode présentation ; rapprochement local ; exposition ; déclenchement par un site qui pèse pour l'utilisateur ; verdict personnel ; signal fournisseur ; concentration cachée et plan B ; colonnes « Marché » et « Ma matière » de l'écran « Fiabilité » ; alertes sans donnée privée (`docs/SPEC.md`, section 11).

Critères : les cas « Ma chaîne » de `docs/SPEC.md` (section 13) passent ; aucune valeur canari ne sort (prompts, requêtes, alertes, journaux, exports publics) ; coffre fermé, aucune réponse hors de `/api/private/` ne trahit Ma chaîne ; tests de phase 4 au vert.

Phase 5 — Mises à jour automatiques et durcissement

Workflow de release (build sans secret, approbation, signature isolée) ; updater avec vérification, test de santé et retour arrière ; première installation par `goulot dev bootstrap` ; LaunchAgents ; page « Bilan sécurité » ; Dependabot avec 7 jours de recul et proposition de version hebdomadaire.

Critères : une archive modifiée ou signée par une clé inconnue est refusée ; une version qui échoue au test de santé est annulée automatiquement ; un retour à une version plus ancienne hors procédure est refusé ; tests de `docs/UPDATES.md` au vert.

Phase 6 — Export de conformité

Modèles de `config/compliance/` (« Article 24 », « Questionnaire client ») ; calculs de concentration (part du premier pays, indice de Herfindahl-Hirschman, repère de 65 %) ; écran « Conformité » avec aperçu et contrôle de complétude ; cartographie fondée sur la chaîne type ou sur Ma chaîne (coffre ouvert, mention « Confidentiel ») ; PDF par l'impression du navigateur, annexes JSON et CSV ; empreinte en pied de page et entrée `export` au registre. Vérifie d'abord le texte en vigueur de l'article 24 et note les écarts dans `docs/DECISIONS.md`.

Critères : un rapport complet pour les 3 CAS de test ; indice de concentration et repère de 65 % justes sur un jeu de parts connu ; un rapport incomplet porte « brouillon » sur chaque page ; un rapport fondé sur Ma chaîne exige le coffre ouvert et n'en laisse aucune copie en clair ; aucun appel réseau ni appel IA pendant l'export ; tests de phase 6 au vert.

Demande toujours à l'utilisateur avant de

- créer le dépôt GitHub (recommande-le public, en une phrase, et note son choix) ou pousser du code ;
- installer ou modifier un LaunchAgent ;
- dépenser plus de 1 $ d'API dans un test ;
- modifier le chiffrement, la hiérarchie des clés, le coffre ou le format du registre ;
- ajouter une dépendance hors de la liste « Stack » ;

- élargir le bac à sable de Claude Code (dossier ou domaine supplémentaire) ;
- changer un réglage macOS (pare-feu, FileVault) : propose, c'est lui qui fait.

Définition de « terminé »

Tests et lint au vert, `tests/security` au vert, docs à jour (`docs/DECISIONS.md` si une décision a été prise), aucun secret ajouté, impact sur le coût IA indiqué.